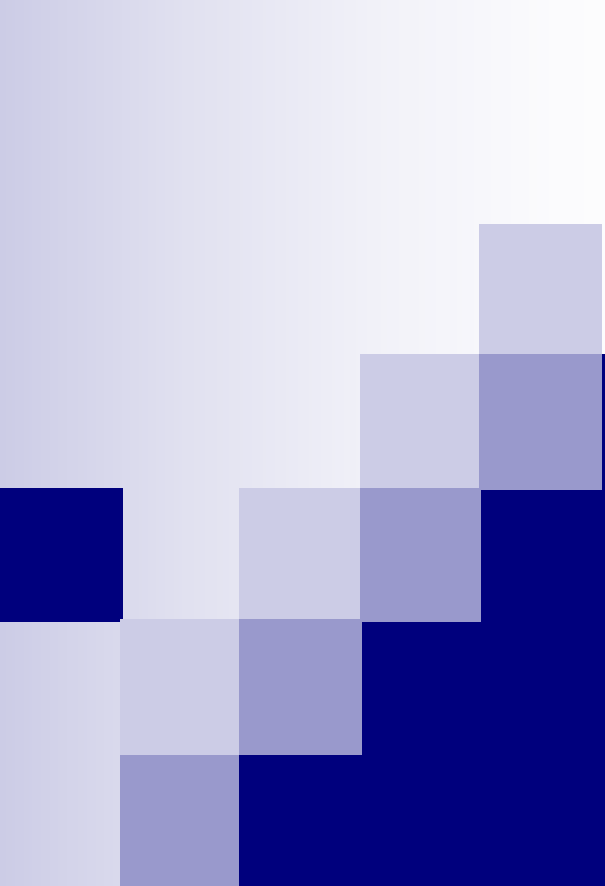




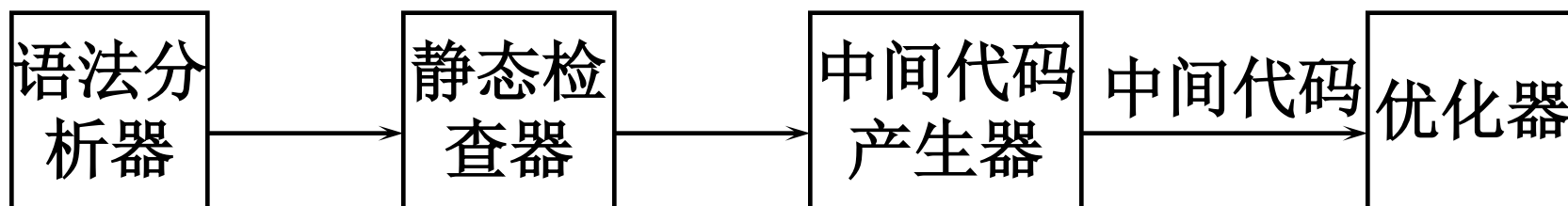
# 第七章 语义分析和 中间代码产生

同济大学计算机系

# 概述

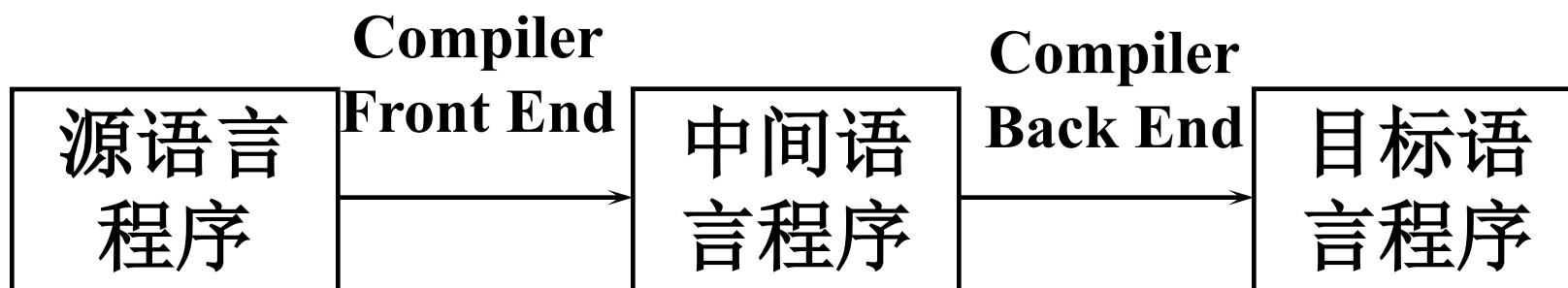
## ■ 静态语义检查

- 类型检查
- 控制流检查
- 一致性检查
- 相关名字检查
- 名字的作用域分析



■ 中间语言（复杂性界于源语言和目标语言之间）的好处：

- 便于进行与机器无关的代码优化工作
- 易于移植
- 使编译程序的结构在逻辑上更为简单明确



# 内容线索

- 中间语言
- 说明语句
- 赋值语句的翻译
- 布尔表达式的翻译
- 控制语句的翻译
- 过程调用的处理

# 中间语言

- 常用的中间语言
  - 后缀式，逆波兰表示
  - 图表示
    - **DAG**
    - 抽象语法树
  - 三地址代码
    - 三元式
    - 四元式
    - 间接三元式

# 后缀式

- **后缀式**表示法：**Lukasiewicz**发明的一种表示表达式的方法，又称**逆波兰**表示法。
- 一个表达式**E**的后缀形式可以如下定义：
  1. 如果**E**是一个变量或常量，则**E**的后缀式是**E**自身。
  2. 如果**E**是 **$E_1$  op  $E_2$** 形式的表达式，其中**op**是任何二元操作符，则**E**的**后缀式**为 **$E_1'$   $E_2'$  op**，其中 **$E_1'$** 和 **$E_2'$** 分别为 **$E_1$** 和 **$E_2$** 的后缀式。
  3. 如果**E**是 **$(E_1)$** 形式的表达式，则 **$E_1$** 的后缀式就是**E**的后缀式。

- 逆波兰表示法不用括号。只要知道每个算符的目数，对于后缀式，不论从哪一端进行扫描，都能对它进行唯一分解。
- 后缀式的计算
  - 用一个栈实现。
  - 一般的计算过程是：自左至右扫描后缀式，每碰到运算量就把它推进栈。每碰到 $k$ 目运算符就把它作用于栈顶的 $k$ 个项，并用运算结果代替这 $k$ 个项。

## 把表达式翻译成后缀式的语义规则描述

| 产生式                                         | 语义规则                                                                                     |
|---------------------------------------------|------------------------------------------------------------------------------------------|
| $E \rightarrow E^{(1)} \text{ op } E^{(2)}$ | $E.\text{code} := E^{(1)}.\text{code} \parallel E^{(2)}.\text{code} \parallel \text{op}$ |
| $E \rightarrow (E^{(1)})$                   | $E.\text{code} := E^{(1)}.\text{code}$                                                   |
| $E \rightarrow \text{id}$                   | $E.\text{code} := \text{id}$                                                             |

- **E.code**表示**E**后缀形式
- **op**表示任意二元操作符
- “**||**”表示后缀形式的连接

$E \rightarrow E^{(1)}op E^{(2)}$

$E \rightarrow (E^{(1)})$

$E \rightarrow id$

$E.code := E^{(1)}.code \parallel E^{(2)}.code \parallel op$

$E.code := E^{(1)}.code$

$E.code := id$

- 数组**POST**存放后缀式：**k**为下标，初值为1
- 上述语义动作可实现为：

产生式

程序段

$E \rightarrow E^{(1)}op E^{(2)}$

$\{POST[k] := op; k := k + 1\}$

$E \rightarrow (E^{(1)})$

$\{\}$

$E \rightarrow i$

$\{POST[k] := i; k := k + 1\}$

- 例：输入串**a+b+c**的分析和翻译

**POST:**

| 1 | 2 | 3 | 4 | 5 |     |
|---|---|---|---|---|-----|
| a | b | + | c | + | ... |

# 图表示法

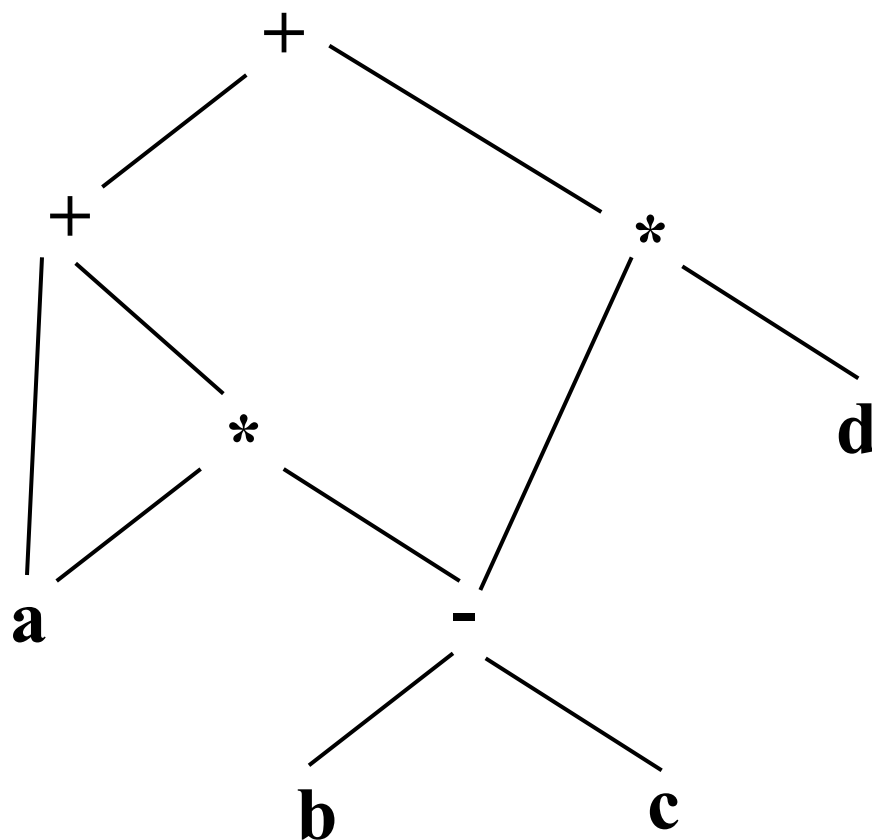
- DAG
- 抽象语法树

# DAG

- 有向无循环图(**Directed Acyclic Graph**, 简称 **DAG**)
  - 对表达式中的每个子表达式, **DAG**中都有一个结点
  - 一个内部结点代表一个操作符, 它的孩子代表操作数
  - 在一个**DAG**中代表公共子表达式的结点具有多个父结点

# $a+a^*(b-c)+(b-c)^*d$ 的图表示法

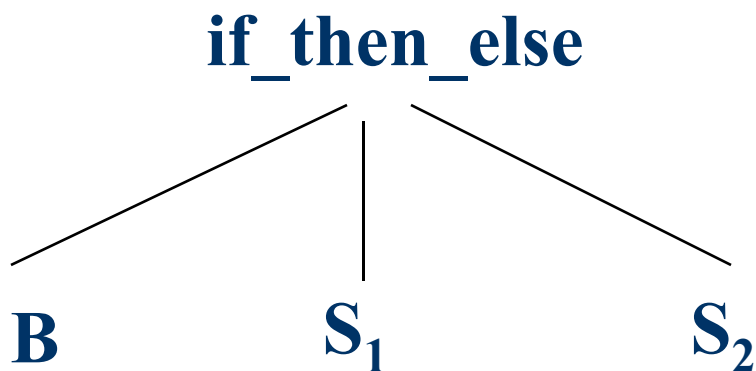
后缀表达式:  $aabc-^*+bc-d^*+$



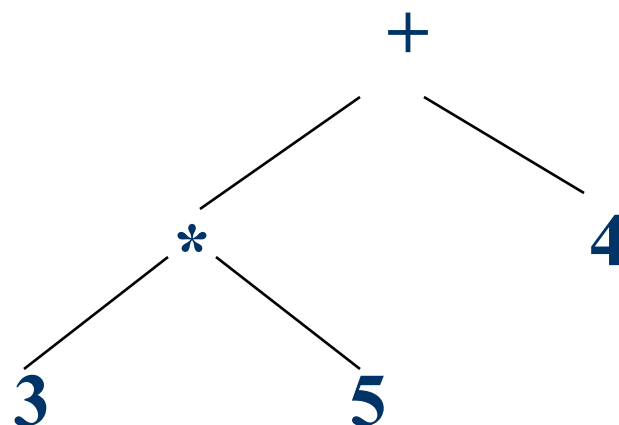
# 抽象语法树

- 在语法树中去掉那些对翻译不必要的信息，从而获得更有效的源程序中间表示。这种经变换后的语法树称之为**抽象语法树**(Abstract Syntax Tree)

□  $S \rightarrow \text{if } B \text{ then } S_1 \text{ else } S_2$



□  $3 * 5 + 4$



# 建立表达式的抽象语法树

- **mknnode (op,left,right)** 建立一个运算符结点，标号是**op**，两个域**left**和**right**分别指向左子树和右子树。
- **mkleaf (id,entry)** 建立一个标识符结点，标号为**id**，一个域**entry**指向标识符在符号表中的入口。
- **mkleaf (num,val)** 建立一个数结点，标号为**num**，一个域**val**用于存放数的值。

# 建立抽象语法树的语义规则

产生式

语义规则

$E \rightarrow E_1 + T$

$E.nptr := \text{mknode} ( '+' , E_1.nptr, T.nptr )$

$E \rightarrow E_1 - T$

$E.nptr := \text{mknode} ( '-' , E_1.nptr, T.nptr )$

$E \rightarrow T$

$E.nptr := T.nptr$

$T \rightarrow (E)$

$T.nptr := E.nptr$

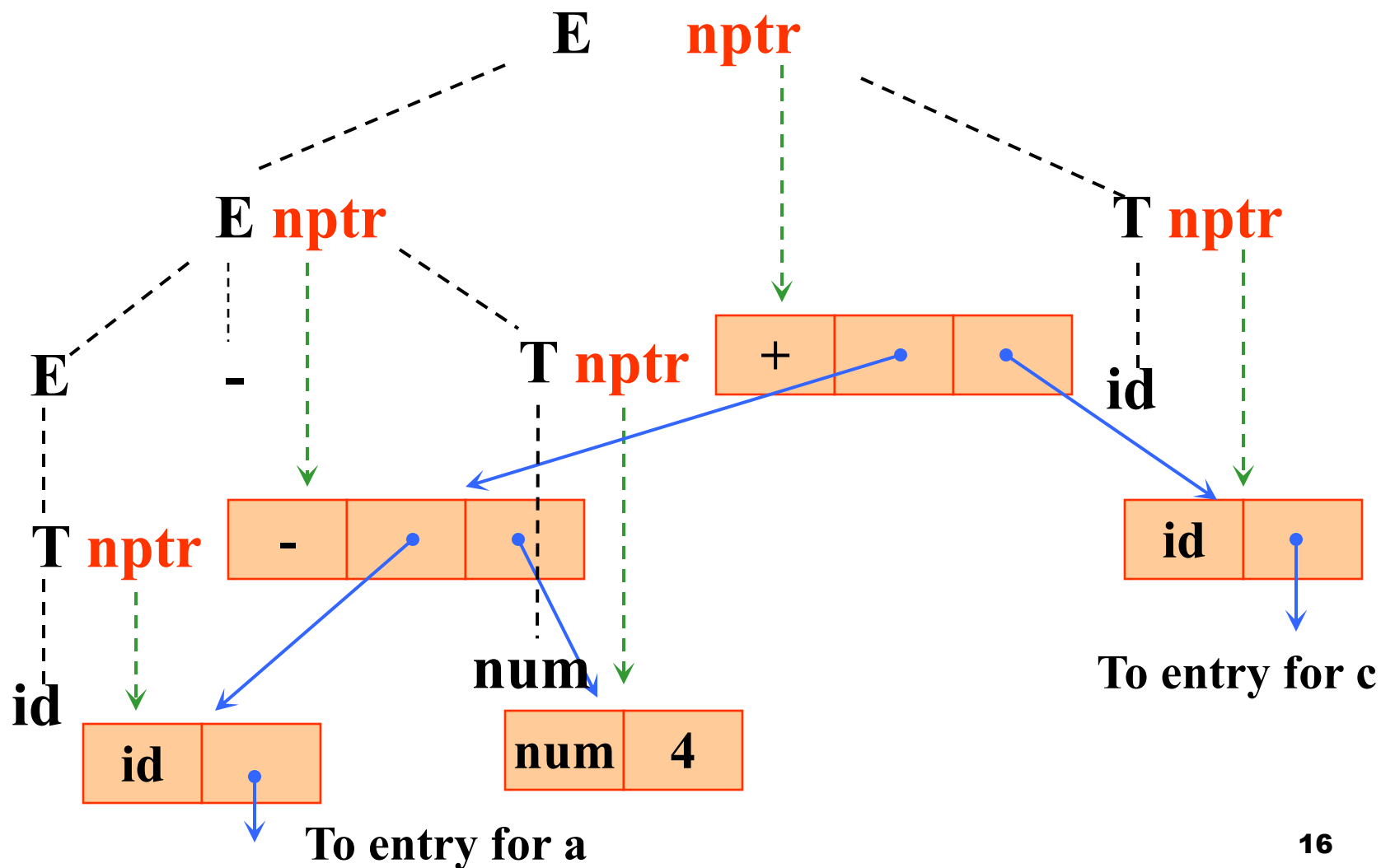
$T \rightarrow \text{id}$

$T.nptr := \text{mkleaf} ( \text{id}, \text{id.entry} )$

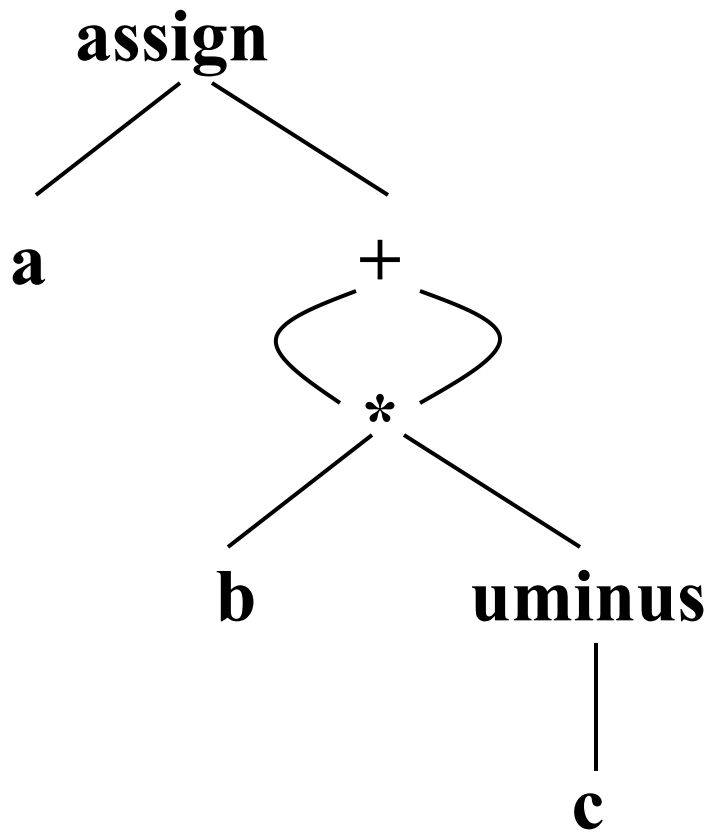
$T \rightarrow \text{num}$

$T.nptr := \text{mkleaf} ( \text{num}, \text{num.val} )$

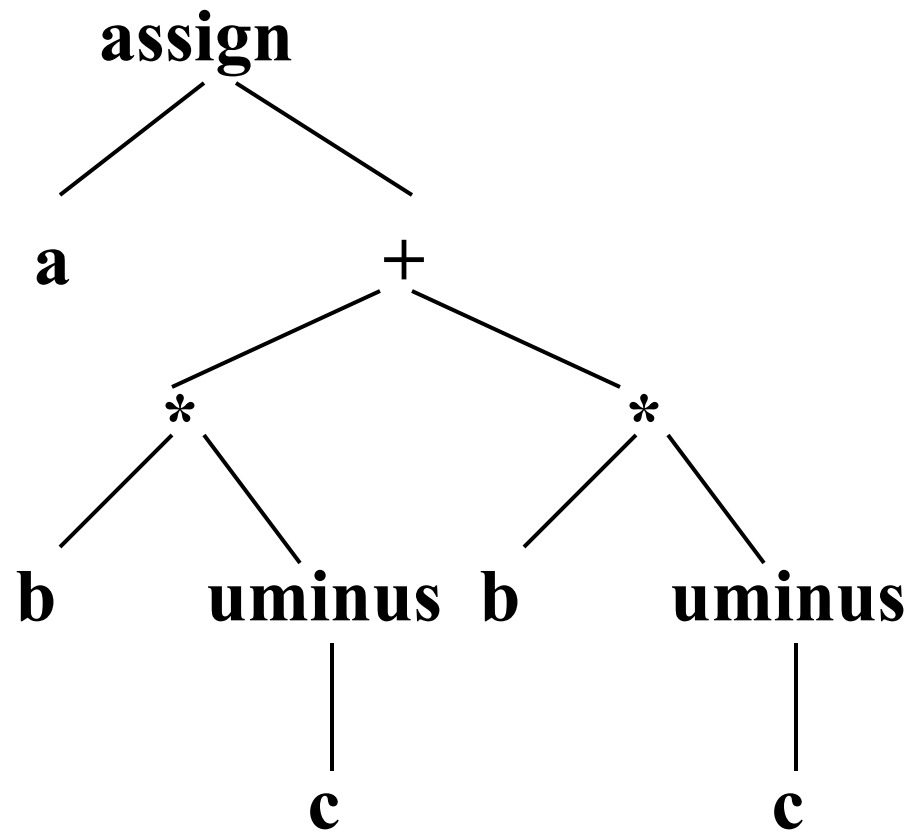
# a - 4 + c 的抽象语法树



# $a := b * (-c) + b * (-c)$ 的图表示法



**DAG**



**抽象语法树**

# 产生赋值语句抽象语法树的属性文法

产生式

语义规则

|                           |                                                                 |
|---------------------------|-----------------------------------------------------------------|
| $S \rightarrow id := E$   | $S.nptr := mknode('assign',$<br>$mkleaf(id, id.place), E.nptr)$ |
| $E \rightarrow E_1 + E_2$ | $E.nptr := mknode('+', E_1.nptr, E_2.nptr)$                     |
| $E \rightarrow E_1 * E_2$ | $E.nptr := mknode('*', E_1.nptr, E_2.nptr)$                     |
| $E \rightarrow -E_1$      | $E.nptr := mknode('uminus', E_1.nptr)$                          |
| $E \rightarrow (E_1)$     | $E.nptr := E_1.nptr$                                            |
| $E \rightarrow id$        | $E.nptr := mkleaf(id, id.place)$                                |

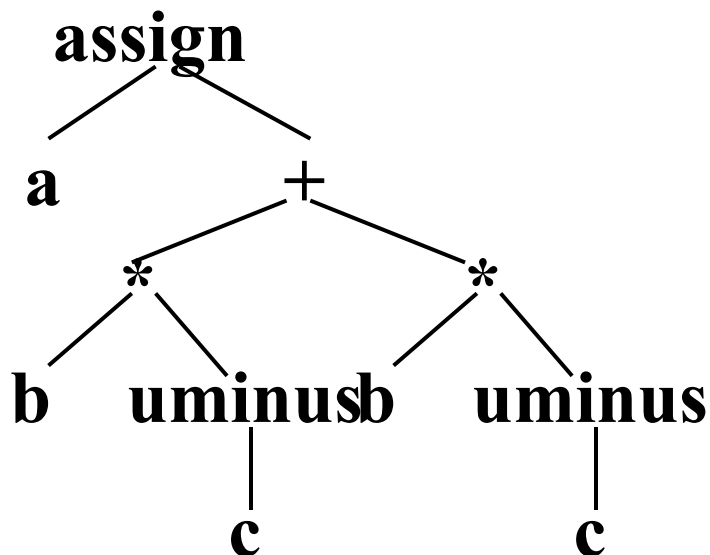
# 三地址代码

- 三地址代码

**$x := y \text{ op } z$**

- 三地址代码可以看成是抽象语法树或 **DAG** 的一种线性表示

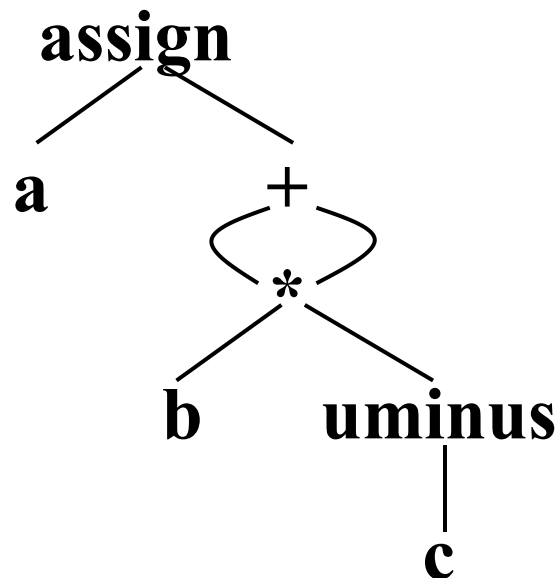
# $a := b * (-c) + b * (-c)$ 的图表示法



抽象语法树

抽象语法树对应的代码:

```
T1 := -c  
T2 := b * T1  
T3 := -c  
T4 := b * T3  
T5 := T2 + T4  
a := T5
```



DAG

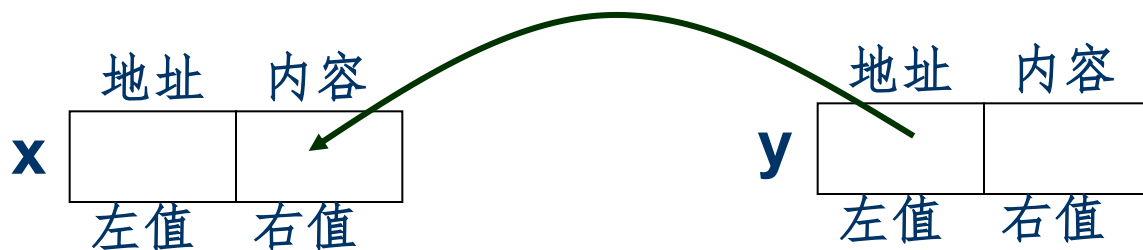
DAG对应的代码:

```
T1 := -c  
T2 := b * T1  
T5 := T2 + T2  
a := T5
```

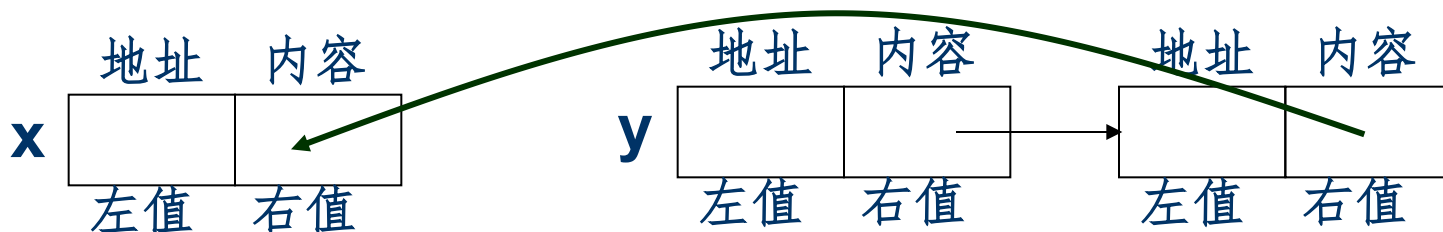
# 三地址语句的种类

- $x := y \text{ op } z$
- $x := \text{op } y$
- $x := y$
- `goto L`
- `if x relop y goto L`或`if a goto L`
- `param x`和`call p,n`，以及返回语句`return y`
- $x := y[i]$ 及 $x[i] := y$ 的索引赋值
- $x := \&y$ ,  $x := *y$ 和 $*x := y$ 的地址和指针赋值

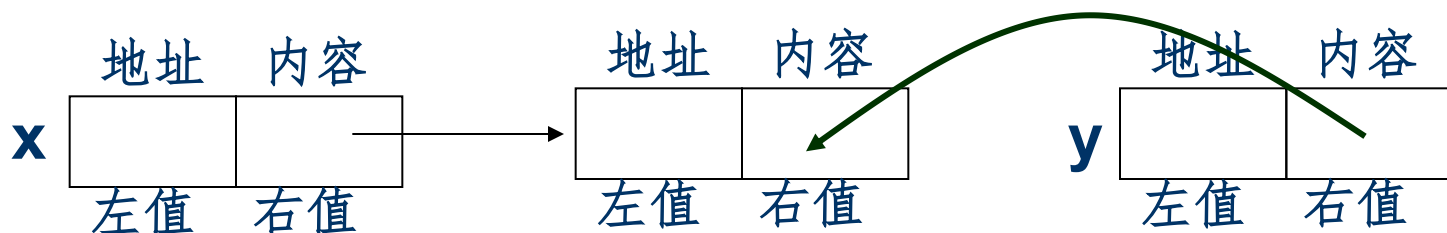
- $x := \&y$  把  $y$  的地址赋给  $x$ ，其中  $y$  是一个名字，或者是一个临时变量，代表一个具有左值的表达式



- $x := *y$  把  $y$  所指示的地址单元里存放的内容赋给  $x$ ，其中  $y$  是一个指针或者是一个其右值为地址的临时变量。



- $*x := y$  把  $y$  的右值赋给  $x$  所指向的对象的右值



# 语句的三地址代码

- 生成三地址代码时，临时变量的名字对应抽象语法树的内部结点
  - 产生式  $E \rightarrow E_1 + E_2$ 
    - $E$  的值放到一个新的临时变量  $T$  中
  - 赋值语句  $id := E$  的三地址代码包括：
    - (1) 对表达式  $E$  求值并置于变量  $T$  中
    - (2) 进行赋值  $id.place := T$

# 赋值语句的文法

产生式

$S \rightarrow id := E$

$E \rightarrow E_1 + E_2$

$E \rightarrow E_1 * E_2$

$E \rightarrow -E_1$

$E \rightarrow (E_1)$

$E \rightarrow id$

- 非终结符号 **S** 有综合属性 **S.code**，它代表赋值语句 **S** 的三地址代码。
- 非终结符号 **E** 有如下两个属性：
  - **E.place** 表示存放 **E** 值的名字。
  - **E.code** 表示对 **E** 求值的三地址语句序列。
  - 函数 **newtemp** 的功能是，每次调用它时，将返回一个不同临时变量名字，如 **T<sub>1</sub>, T<sub>2</sub>, …**。
  - **gen** 表示生成三地址语句
- 三地址语句序列往往是被存放到一个输出文件中

# S-属性文法定义

| 产生式                       | 语义规则                                                                                                                                                      |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| $S \rightarrow id := E$   | $S.code := E.code \parallel \text{gen}(id.place \text{ ':=' } E.place)$                                                                                   |
| $E \rightarrow E_1 + E_2$ | $E.place := \text{newtemp};$<br>$E.code := E_1.code \parallel E_2.code \parallel$<br>$\text{gen}(E.place \text{ ':=' } E_1.place \text{ '+' } E_2.place)$ |
| $E \rightarrow E_1 * E_2$ | $E.place := \text{newtemp};$<br>$E.code := E_1.code \parallel E_2.code \parallel$<br>$\text{gen}(E.place \text{ ':=' } E_1.place \text{ '*' } E_2.place)$ |
| $E \rightarrow -E_1$      | $E.place := \text{newtemp};$<br>$E.code := E_1.code \parallel$<br>$\text{gen}(E.place \text{ ':=' 'uminus' } E_1.place)$                                  |
| $E \rightarrow (E_1)$     | $E.place := E_1.place;$<br>$E.code := E_1.code$                                                                                                           |
| $E \rightarrow id$        | $E.place := id.place;$<br>$E.code = \text{' '}$                                                                                                           |

# 三地址语句

$a := b * (-c) + b * (-c)$

## ■ 四元式

- 一个带有四个域的记录结构，这四个域分别称为 **op**, **arg1**, **arg2** 及 **result**

|     | op     | arg1           | arg2           | result         |
|-----|--------|----------------|----------------|----------------|
| (0) | uminus | c              |                | T <sub>1</sub> |
| (1) | *      | b              | T <sub>1</sub> | T <sub>2</sub> |
| (2) | uminus | c              |                | T <sub>3</sub> |
| (3) | *      | b              | T <sub>3</sub> | T <sub>4</sub> |
| (4) | +      | T <sub>2</sub> | T <sub>4</sub> | T <sub>5</sub> |
| (5) | :=     | T <sub>5</sub> |                | a              |

- 四元式之间的联系通过临时变量实现。
- 单目运算只用 **arg1** 域，转移语句将目标标号放入 **result** 域。
- **arg1, arg2, result** 通常为指针，指向有关名字的符号表入口，且临时变量填入符号表。

# 三地址语句

$a := b * (-c) + b * (-c)$

## ■ 三元式

- 通过计算临时变量值的语句的位置来引用这个临时变量
- 三个域：**op**、**arg1**和**arg2**

|     | <b>op</b>     | <b>arg1</b> | <b>arg2</b> |
|-----|---------------|-------------|-------------|
| (0) | <b>uminus</b> | <b>c</b>    |             |
| (1) | <b>*</b>      | <b>b</b>    | (0)         |
| (2) | <b>uminus</b> | <b>c</b>    |             |
| (3) | <b>*</b>      | <b>b</b>    | (2)         |
| (4) | <b>+</b>      | (1)         | (3)         |
| (5) | <b>assign</b> | <b>a</b>    | (4)         |

# 三地址语句

- 多目运算的三元式

例.  $x[i] := y$

|     | op     | arg1 | arg2 |
|-----|--------|------|------|
| (0) | [ ] =  | x    | i    |
| (1) | assign | (0)  | y    |

例.  $x := y[i]$

|     | op     | arg1 | arg2 |
|-----|--------|------|------|
| (0) | = [ ]  | y    | i    |
| (1) | assign | x    | (0)  |

# 三地址语句

## ■ 间接三元式

- 为了便于代码优化，用三元式表+间接码表表示中间代码
  - 间接码表：一张指示器表，按运算的先后次序列出有关三元式在三元式表中的位置。
- 优点：方便优化，节省空间

- 例如，语句

**$X := (A + B) * C;$**

**$Y := D \uparrow (A + B)$**

的间接三元式表示如下表所示。

间接代码

(1)

(2)

(3)

(1)

(4)

(5)

三元式表

|     | OP | ARG1 | ARG2 |
|-----|----|------|------|
| (1) | +  | A    | B    |
| (2) | *  | (1)  | C    |
| (3) | := | X    | (2)  |
| (4) | ↑  | D    | (1)  |
| (5) | := | Y    | (4)  |

## 四元式、三元式和间接三元式比较

- 三元式中使用了指向三元式的指针，优化时修改较难。
- 间接三元式优化只需要更改间接码表，并节省三元式表存储空间。
- 修改四元式表也较容易，只是临时变量要填入符号表，占据一定存储空间。

# 内容线索

- ✓ 中间语言
  - 说明语句的翻译
  - 赋值语句的翻译
  - 布尔表达式的翻译
  - 控制语句的翻译
  - 过程调用的处理

# 概述

- 说明部分中把定义性出现的标识符与类型等属性相关联,从而确定它们在计算机内部的表示法、取值范围及可对其进行的运算。
- 为了产生有效地可执行目标代码,对于说明部分的翻译,不仅仅把与标识符相关联的类型等属性填入符号表中,还必须考虑到标识符所标记的对象的存储分配问题。

# 常量定义的翻译

- C++语言中有常量定义，以关键字**const**为标志，如  
**const float pi=3.1416, one=1.0**
- 对每个常量定义的处理应包括下列工作：
  - (1) 把等号右边的常量值登录入常量表中（若之前尚未登录）并回送常量表序号；
  - (2) 然后为等号左边的标识符建立符号表新条目，在该条目中填入常量标志、相应类型及常量表序号。

- 假定常量定义中不指明类型，类型直接由常量值本身确定，且等号右边仅整数或常量标识符。
- 常量定义及相应翻译方案

**CONSTEDF** → **const CDT**

**CDT** → **CDT, CD**

**CDT** → **CD**

**CD** → **id=num**

**{ num.ord:=lookCT(num.lexval)  
id.ord:=num.ord; id.type:=integer;  
id.kind:=CONSTANT  
add(id.entry, id.kind, id.type, id.ord)}**

**CD** → **id=id<sub>1</sub>**

**{id.kind:=CONSTANT; id.type:=id<sub>1</sub>.type;  
id.ord:=id<sub>1</sub>.ord  
add(id.entry, id.kind, id.type, id.ord)}**

**lookCT(c)**将在常量表中查找常量**c**，若查不到，则把给常量值登录入常量表。最终回送常量**c**值在常量表中的序号

过程**add**是对过程**addtype**的扩充，它把种类、类型与序号三者填入符号表相应条目中

# 变量说明的翻译

- 变量说明部分由一组变量说明语句组成，这里假定每个变量说明语句中仅包含一个标识符。
- 变量说明部分的语法定义

$P \rightarrow D;$

$D \rightarrow D; D$

$D \rightarrow \text{id}: T$

$T \rightarrow \text{integer}$

$T \rightarrow \text{real}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$

$T \rightarrow \uparrow T_1$

## ■ 变量说明部分的翻译

$P \rightarrow D; \quad \{\text{offset}:=0\}$

$D \rightarrow D; D$

$D \rightarrow \text{id}: T$

$\{\text{enter}(\text{id.name}, T.\text{type}, \text{offset});$   
 $\text{offset}:=\text{offset}+T.\text{width}\}$

$T \rightarrow \text{integer}$

$\{T.\text{type}:=\text{integer}; T.\text{width}:=4\}$

$T \rightarrow \text{real}$

$\{T.\text{type}:=\text{real}; T.\text{width}:=8\}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$

$\{T.\text{type}:=\text{array}(\text{num.val}, T_1.\text{type});$   
 $T.\text{width}:=\text{num.val}*T_1.\text{width}\}$

$T \rightarrow \uparrow T_1$

$\{T.\text{type}:=\text{pointer}(T_1.\text{type}); T.\text{width}:=4\}$

# 文法转换

## ■ $P \rightarrow D; \text{\textcolor{red}\{offset:=0\}}$

在处理第一条说明语句之前，需要对**offset**赋初值；

为了使得**offset**赋初值更明显

变更为  $P \rightarrow \text{\textcolor{red}\{offset:=0\}} D;$

也可采用增加非终结符号及产生式来实现

$$P \rightarrow M D$$
$$M \rightarrow \varepsilon \text{\textcolor{red}\{offset:=0\}}$$

# 保留作用域信息

- 过程（函数）定义的语法

$P \rightarrow D;$

$D \rightarrow D; D$

$D \rightarrow \text{id}: T$

$D \rightarrow \text{proc id}; D; S$

$T \rightarrow \text{integer}$

$T \rightarrow \text{real}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1$

$T \rightarrow \uparrow T_1$

- 过程及函数定义的翻译必然涉及标识符作用域问题

- 基本思想：每个过程有一张独立的符号表

## ■ 过程示例

```
(1) Program sort(input, output)
(2)  var a: array[0..10] of integer;
(3)      x: integer;
(4)      procedure readarray;
(5)          var i: integer;
(6)          begin ...a ... end {readarray}
(7)      procedure exchange(i, j:integer)
(8)          begin
(9)              x:=a[i]; a[i]:=a[j]; a[j]:=x
(10)         end {exchange}
(11)     procedure quicksort (m, n:integer);
(12)         var k, v: integer;
(13)         function partition (y,z: integer): integer;
(14)             var i,j: integer;
(15)             begin ...a ...
(16)                 ...v ...
(17)                 ...exchange(i, j); ...
(18)             end {partition}
(19)         begin ... end {quicksort};
(20) begin ... end {sort}.
```

## ■ 语义规则中的操作

- **mktable(previous)** : 创建一张新符号表, 并返回指向新表的一个指针。
  - 参数**previous**指向一张先前创建的符号表, 其值放在新符号表表头
- **enter(table, name, type, offset)**: 在指针**table**指示的符号表中为名字**name**建立一个新项, 并把类型**type**、相对地址**offset**填入到该项中。
- **addwidth(table, with)**: 指针**table**指示的符号表表头中记录下该表中所有名字占用的总宽度
- **enterproc(table, name, newtable)**: 在指针**table**指示的符号表中为名字**name**的过程建立一个新项。
  - 参数**newtable**指向过程**name**的符号表

## ■ 翻译

$P \rightarrow M D;$

{addwidth(top(tblptr), top(offset));  
pop(tblptr); pop(offset)}

$M \rightarrow \varepsilon$

{t:=mktable(nil);  
push(t, tblptr); push(0, offset)}

$D \rightarrow D_1; D_2$

$D \rightarrow \text{proc id}; N D_1; S$

{t:=top(tblptr);  
addwidth(t, top(offset));  
pop(tblptr); pop(offset)  
enterproc(top(tblptr), id.name, t)}

$D \rightarrow \text{id}: T$

{enter(top(tblptr), id.name, T.type,  
top(offset));  
top(offset):=top(offset)+T.width}

$N \rightarrow \varepsilon$

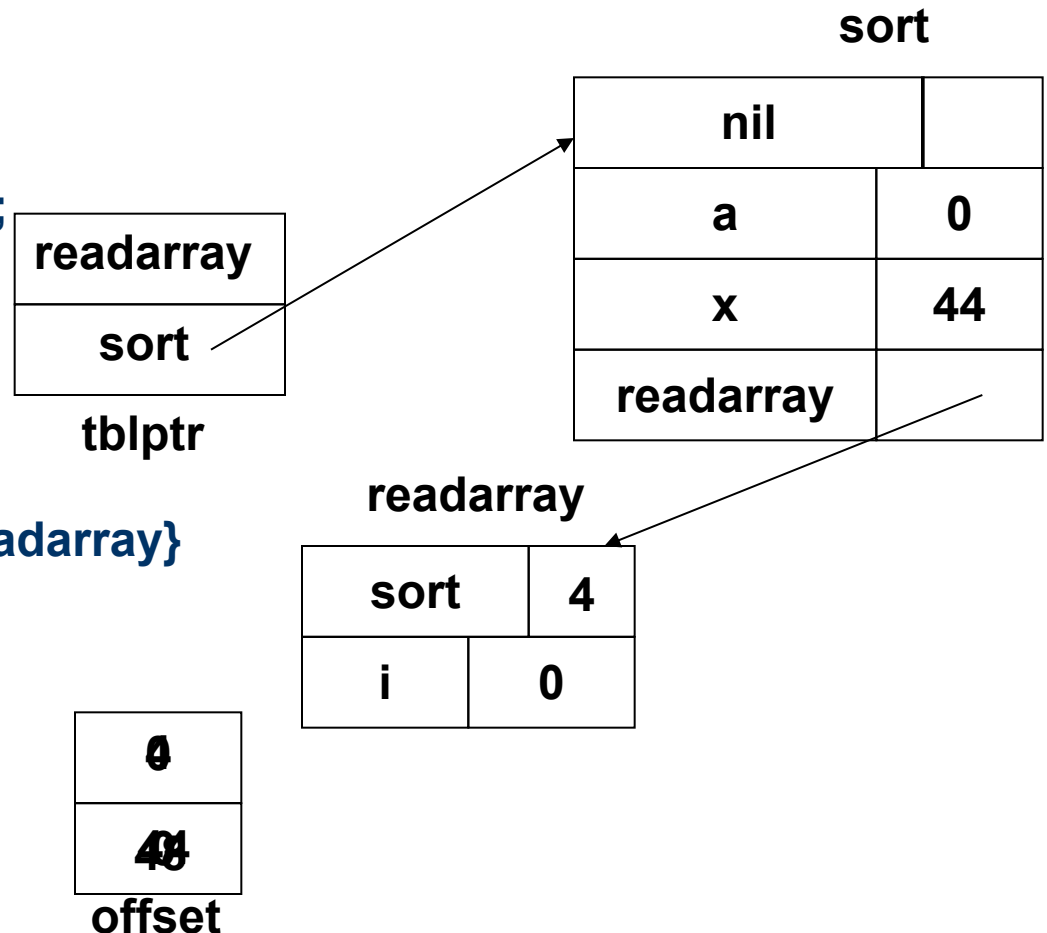
{t:=mktable(top(tblptr));  
push(t, tblptr); push(0, offset)}

- 栈**tblptr** 保存各外层过程的符号表指针
- 栈**offset** 存放各嵌套过程的当前相对地址
  - 栈顶元素为当前被处理过程的下一个名字的相对地址

```

(1) Program sort(input, output)
(2)  var a: array[0..10] of integer;
(3)      x: integer;
(4)      procedure readarray;
(5)          var i: integer;
(6)          begin ...a ... end {readarray}
.....

```



# 内容线索

- ✓ 中间语言
- ✓ 说明语句的翻译
- 赋值语句的翻译
- 布尔表达式的翻译
- 控制语句的翻译
- 过程调用的处理

# 赋值语句的翻译

## ■ 简单算术表达式及赋值语句

### □ 简单算术表达式及赋值语句翻译为三地址代码的翻译模式

- 属性**id.name** 表示**id**所代表的名字本身
- 过程**lookup(id.name)**检查是否在符号表中存在相应此名字的入口。如果有，则返回一个指向该表项的指针，否则，返回**nil**表示没有找到
- 过程**emit**将生成的三地址语句发送到输出文件中

# 产生赋值语句三地址代码的翻译模式

$S \rightarrow id := E$   $S.code := E.code \parallel gen(id.place \text{ ':=' } E.place)$

$E \rightarrow E_1 + E_2$   $E.place := newtemp;$

$E.code := E_1.code \parallel E_2.code \parallel gen(E.place \text{ ':=' } E_1.place \text{ '+' } E_2.place)$

$E \rightarrow E_1 * E_2$   $E.place := newtemp;$

$E.code := E_1.code \parallel E_2.code \parallel gen(E.place \text{ ':=' } E_1.place \text{ '*' } E_2.place)$

```
S → id := E    { p := lookup(id.name);  
                  if p ≠ nil then  
                      emit(p := E.place)  
                  else error }
```

```
E → E1 + E2    { E.place := newtemp;  
                  emit(E.place := E1.place + E2.place) }
```

```
E → E1 * E2    { E.place := newtemp;  
                  emit(E.place := E1.place * E2.place) }
```

|                       |                                                                                                                 |
|-----------------------|-----------------------------------------------------------------------------------------------------------------|
| $E \rightarrow -E_1$  | <b>E.place:=newtemp;</b><br><b>E.code:=E<sub>1</sub>.code    gen(E.place ‘:=’ ‘uminus’ E<sub>1</sub>.place)</b> |
| $E \rightarrow (E_1)$ | <b>E.place:=E<sub>1</sub>.place;</b><br><b>E.code:=E<sub>1</sub>.code</b>                                       |
| $E \rightarrow id$    | <b>E.place:=id.place;</b><br><b>E.code= ‘ ’</b>                                                                 |

```

E → -E1      { E.place:=newtemp;
                  emit(E.place‘:=’‘uminus’E1.place)}

E → (E1)      { E.place:=E1.place}

E → id         { p:=lookup(id.name);
                  if p≠nil then
                      E.place:=p
                  else error }

```

# 类型转换

- 用 **E.type** 表示非终结符 **E** 的类型属性
- 对应产生式 **E** → **E<sub>1</sub> op E<sub>2</sub>** 的语义动作中关于 **E.type** 的语义规则可定义为：  
    { if **E<sub>1</sub>.type=integer** and **E<sub>2</sub>.type=integer**  
        **E.type:=integer**  
    else **E.type:=real** }
- 进行类型转换的三地址代码  
    **x:= inttoreal y**
- 算符区分为整型算符 **int op** 和实型算符 **real op**,

例. **x:=y + i\*j**

其中 **x**、**y** 为实型；**i**、**j** 为整型。这个赋值句产生的三地址代码为：

```
T1:=i int* j
T3:=inttoreal T1
T2:=y real+ T3
x:=T2
```

## 关于产生式 $E \rightarrow E_1 + E_2$ 的语义动作

```
{ E.place:=newtemp;  
  if  $E_1.type=integer$  and  $E_2.type=integer$  then begin  
    emit (E.place ':='  $E_1.place$  'int+'  $E_2.place$ );  
    E.type:=integer  
  end  
  else if  $E_1.type=real$  and  $E_2.type=real$  then begin  
    emit (E.place ':='  $E_1.place$  'real+'  $E_2.place$ );  
    E.type:=real  
  end
```

else if  $E_1.type = \text{integer}$  and  $E_2.type = \text{real}$  then begin

$u := \text{newtemp};$

    emit ( $u \text{ ' := ' 'inttoreal' } E_1.place$ );

    emit ( $E.place \text{ ' := ' } u \text{ 'real+' } E_2.place$ );

$E.type := \text{real}$

end

else if  $E_1.type = \text{real}$  and  $E_2.type = \text{integer}$  then begin

$u := \text{newtemp};$

    emit ( $u \text{ ' := ' 'inttoreal' } E_2.place$ );

    emit ( $E.place \text{ ' := ' } E_1.place \text{ 'real+' } u$ );

$E.type := \text{real}$

end

else  $E.type := \text{type\_error}$ }

# 内容线索

- ✓ 中间语言
- ✓ 说明语句的翻译
- ✓ 赋值语句的翻译
- 布尔表达式的翻译
- 控制语句的翻译
- 过程调用的处理

# 布尔表达式的翻译

- 布尔表达式：用布尔运算符把布尔量、关系表达式联结起来的式子。
  - 布尔运算符： **and, or, not** ;
  - 关系运算符 **<, ≤, =, ≠, >, ≥**
- 布尔表达式的两个基本作用：
  - 用于逻辑演算, 计算逻辑值;
  - 用于控制语句的条件式.
- 产生布尔表达式的文法：
  - $E \rightarrow E \text{ or } E \mid E \text{ and } E \mid \neg E \mid (E) \mid \text{id rop id} \mid \text{id}$
- 运算符优先级：布尔运算由高到低:**not and or**，同级左结合  
关系运算符同级，且高于布尔运算符

■ 计算布尔表达式通常采用两种方法:

(1) 如同计算算术表达式一样,一步步算

$1 \text{ or } (\text{not } 0 \text{ and } 0) \text{ or } 0$

$= 1 \text{ or } (1 \text{ and } 0) \text{ or } 0$

$= 1 \text{ or } 0 \text{ or } 0$

$= 1 \text{ or } 0$

$= 1$

(2) 采用优化措施

把  $A \text{ or } B$  解释成

if A then true else B

把  $A \text{ and } B$  解释成

if A then B else false

把  $\neg A$  解释成

if A then false else true

对应的, 有**两种**布尔表达式翻译方法

# 数值表示法

- **a or b and not c** 翻译成

$T_1 := \text{not } c$

$T_2 := b \text{ and } T_1$

$T_3 := a \text{ or } T_2$

- **a<b**的关系表达式可等价地写成  
if a<b then 1 else 0 , 翻译成

100:      if a<b goto 103

101:      T:=0

102:      goto 104

103:      T:=1

104:

# 数值表示法的翻译模式

- 过程**emit**将三地址代码送到输出文件中
- **nextstat**: 给出输出序列中下一条三地址语句的地址索引
- 每产生一条三地址语句后，过程**emit**便把**nextstat**加1

# 数值表示法的翻译模式

|                                      |                                                                                                                                                  |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| $E \rightarrow E_1 \text{ or } E_2$  | $\{E.\text{place} := \text{newtemp};$<br>$\text{emit}(E.\text{place} \text{ ' := ' } E_1.\text{place} \text{ ' or ' }$<br>$E_2.\text{place})\}$  |
| $E \rightarrow E_1 \text{ and } E_2$ | $\{E.\text{place} := \text{newtemp};$<br>$\text{emit}(E.\text{place} \text{ ' := ' } E_1.\text{place} \text{ ' and ' }$<br>$E_2.\text{place})\}$ |
| $E \rightarrow \text{not } E_1$      | $\{E.\text{place} := \text{newtemp};$<br>$\text{emit}(E.\text{place} \text{ ' := ' 'not' } E_1.\text{place})\}$                                  |
| $E \rightarrow (E_1)$                | $\{E.\text{place} := E_1.\text{place}\}$                                                                                                         |

# 数值表示法的翻译模式

**a<b 翻译成**

```
100:  if a<b goto 103
101:  T:=0
102:  goto 104
103:  T:=1
104:
```

```
E→id1 relop id2 { E.place:=newtemp;
                      emit( 'if' id1.place relop. op
                           id2. place 'goto' nextstat+3);
                      emit(E.place ':=' '0' );
                      emit( 'goto' nextstat+2);
                      emit(E.place ':=' '1' ) }
```

```
E→id { E.place:=id.place }
```

## 布尔表达式 $a < b$ or $c < d$ and $e < f$ 的翻译结果

100: if  $a < b$  goto 103

101:  $T_1 := 0$

102: goto 104

103:  $T_1 := 1$

104: if  $c < d$  goto 107

105:  $T_2 := 0$

106: goto 108

107:  $T_2 := 1$

108: if  $e < f$  goto 111

109:  $T_3 := 0$

110: goto 112

111:  $T_3 := 1$

112:  $T_4 := T_2$  and  $T_3$

113:  $T_5 := T_1$  or  $T_4$

$E \rightarrow id_1 \text{ relop } id_2$

```
{ E.place:=newtemp;
  emit( 'if'  id1.place relop. op id2. place
        'goto' nextstat+3);
  emit(E.place ':=' '0' );
  emit( 'goto' nextstat+2);
  emit(E.place ':=' '1' ) }
```

$E \rightarrow id$

```
{ E.place:=id.place }
```

$E \rightarrow E_1 \text{ or } E_2$

```
{ E.place:=newtemp;
  emit(E.place ':=' E1.place 'or' E2.place)}
```

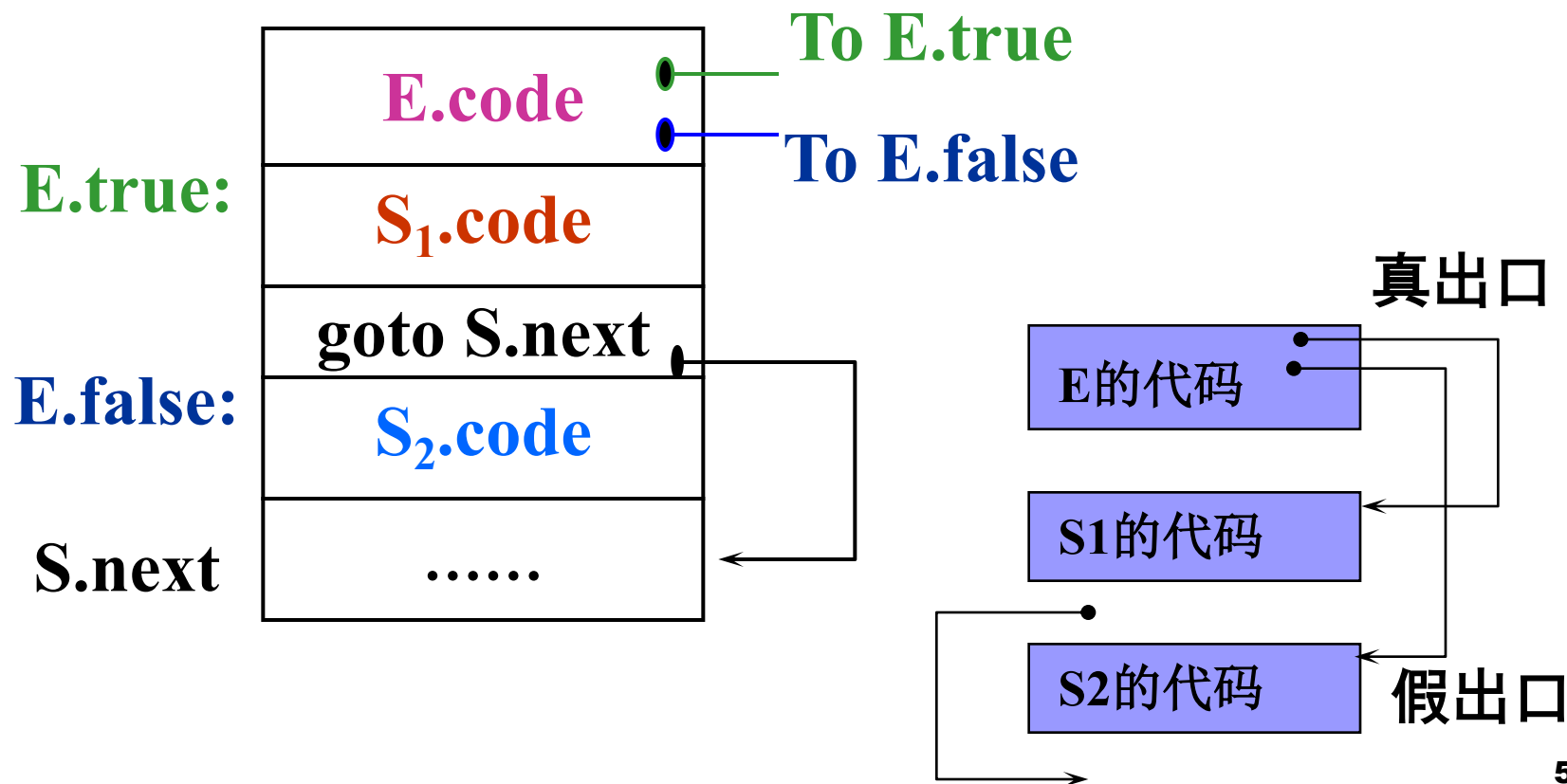
$E \rightarrow E_1 \text{ and } E_2$

```
{ E.place:=newtemp;
  emit(E.place ':=' E1.place 'and' E2.place) }
```

# 作为条件控制的布尔式翻译

## ■ 条件语句 if E then S<sub>1</sub> else S<sub>2</sub>

赋予 E 两种出口:一真一假



例. 把语句: **if a>c or b <d then S<sub>1</sub> else S<sub>2</sub>**  
翻译成如下的一串三地址代码

**if a>c goto L2** “真” 出口

**goto L1** “真” 出口

**L1: if b<d goto L2** “假” 出口

**goto L3**

**L2:** (关于**S<sub>1</sub>**的三地址代码序列)  
**goto Lnext**

**L3:** (关于**S<sub>2</sub>**的三地址代码序列)

**Lnext:**

# 产生布尔表达式三地址代码的语义规则

产生式

$E \rightarrow E_1 \text{ or } E_2$

语义规则

每次调用函数  
newlabel后都返回  
一个新的符号标号

E.true是E为‘真’  
时控制流转向的标  
号

$E_1.\text{true} := E.\text{true};$   
 $E_1.\text{false} := \text{newlabel};$   
 $E_2.\text{true} := E.\text{true};$   
 $E_2.\text{false} := E.\text{false};$   
 $E.\text{code} := E_1.\text{code} \parallel$   
 $\text{gen}(E_1.\text{false} \text{ ‘:’ }) \parallel E_2.\text{code}$

E.false是E为‘假’  
时控制流转向的标  
号

$E_1.\text{code}$  — To E.true

To  $E_1.\text{false}$

$E_2.\text{code}$  — To E.true

To E.false

# 产生布尔表达式三地址代码的语义规则

产生式

语义规则

$E \rightarrow E_1 \text{ and } E_2$

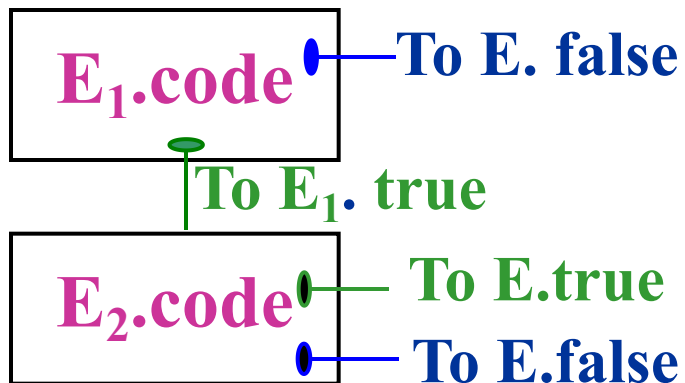
$E_1.\text{true} := \text{newlabel};$

$E_1.\text{false} := E.\text{false};$

$E_2.\text{true} := E.\text{true};$

$E_2.\text{false} := E.\text{false};$

$E.\text{code} := E_1.\text{code} \parallel$   
 $\text{gen}(E_1.\text{true} \text{ ':' } ) \parallel E_2.\text{code}$



# 产生布尔表达式三地址代码的语义规则

产生式

语义规则

$E \rightarrow \text{not } E_1$

$E_1.\text{true} := E.\text{false};$   
 $E_1.\text{false} := E.\text{true};$   
 $E.\text{code} := E_1.\text{code}$

$E \rightarrow (E_1)$

$E_1.\text{true} := E.\text{true};$   
 $E_1.\text{false} := E.\text{false};$   
 $E.\text{code} := E_1.\text{code}$

# 产生布尔表达式三地址代码的语义规则

产生式

语义规则

$E \rightarrow id_1 \text{ relop } id_2$      $E.code := \text{gen}(\text{'if' } id_1.place \text{ relop.op } id_2.place \text{'goto' } E.true) \parallel \text{gen}(\text{'goto' } E.false)$

$E \rightarrow \text{true}$                        $E.code := \text{gen}(\text{'goto' } E.true)$

$E \rightarrow \text{false}$                        $E.code := \text{gen}(\text{'goto' } E.false)$

考虑如下表达式：

**$a < b$  or  $c < d$  and  $e < f$**

假定整个表达式的真假出口已分别置为**Ltrue**和**Lfalse**，则按定义将生成如下的代码：

**if  $a < b$  goto Ltrue**

**goto  $L_1$**

**$L_1$ :** **if  $c < d$  goto  $L_2$**

**goto Lfalse**

**$L_2$ :** **if  $e < f$  goto Ltrue**

**goto Lfalse**

# 布尔表达式的翻译

## ■ 两遍扫描

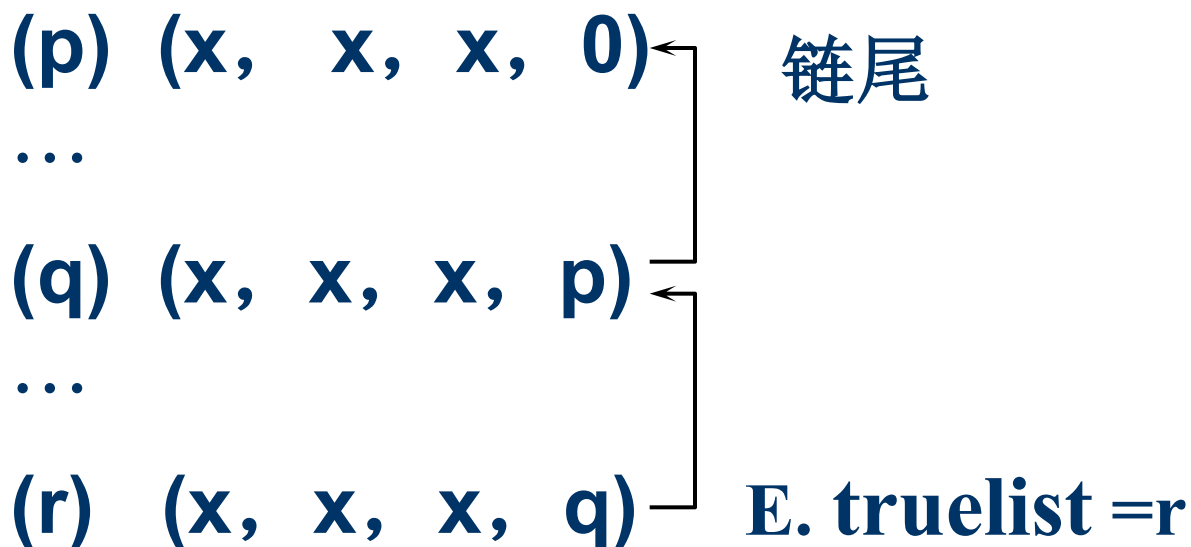
- 为给定的输入串构造一棵语法树；
- 对语法树进行深度优先遍历，进行语义规则中规定的翻译。

## ■ 一遍扫描

# 一遍扫描实现布尔表达式的翻译

- 采用四元式形式
- 把四元式存入一个数组中，数组下标就代表四元式的标号
- 约定
  - 四元式(jnz, a, -, p) 表示 if a goto p
  - 四元式(jrop, x, y, p)表示 if x rop y goto p
  - 四元式(j, -, -, p) 表示 goto p
- 有时,四元式转移地址无法立即知道,我们只好把这个未完成的四元式地址作为E的语义值保存,待机"回填"。

- 为非终结符 **E** 赋予两个综合属性 **E.truelist** 和 **E.falselist**。它们分别记录布尔表达式 **E** 所对应的四元式中需回填“真”、“假”出口的四元式的标号所构成的链表
- 例如:假定 **E** 的四元式中需要回填"真"出口的 **p**, **q**, **r** 三个四元式, 则 **E.truelist** 为下列链:



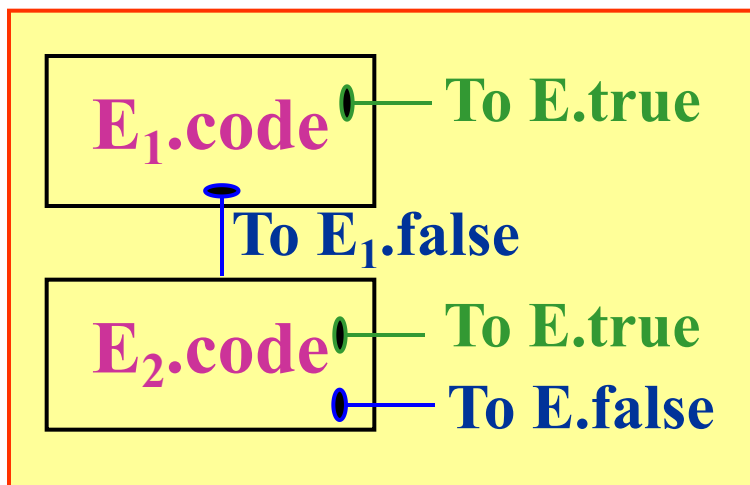
## ■ 为了处理 **E.truelist** 和 **E.falselist**，引入下列语义变量和过程：

- 变量 **nextquad**，它指向下一条将要产生但尚未形成的四元式的地址(标号)。**nextquad**的初值为1，每当执行一次 **emit** 之后，**nextquad** 将自动增1。
- 函数 **makelist(i)**，它将创建一个仅含 **i** 的新链表，其中 **i** 是四元式数组的一个下标(标号)；函数返回指向这个链的指针。
- 函数 **merge(p<sub>1</sub>, p<sub>2</sub>)**，把以 **p<sub>1</sub>** 和 **p<sub>2</sub>** 为链首的两条链合并为一，作为函数值，回送合并后的链首。
- 过程 **backpatch(p, t)**，其功能是完成“回填”，把 **p** 所链接的每个四元式的第四区段都填为 **t**。

# 布尔表达式的文法

- (1)  $E \rightarrow E_1 \text{ or } M E_2$
- (2)  $\quad \quad \quad | E_1 \text{ and } M E_2$
- (3)  $\quad \quad \quad | \text{ not } E_1$
- (4)  $\quad \quad \quad | (E_1)$
- (5)  $\quad \quad \quad | \text{id}_1 \text{ relop id}_2$
- (6)  $\quad \quad \quad | \text{id}$
- (7)  $M \rightarrow \varepsilon$

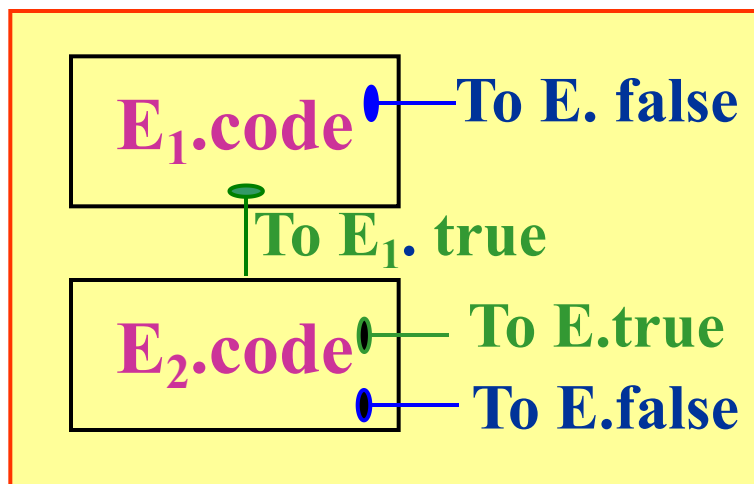
# 布尔表达式的翻译模式



(1)  $E \rightarrow E_1 \text{ or } E_2$

```
{ backpatch( $E_1.falselist$ ,  $M.quad$ );  
   $E.truelist := merge(E_1.truelist, E_2.truelist)$ ;  
   $E.falselist := E_2.falselist$  }
```

# 布尔表达式的翻译模式



(2)  $E \rightarrow E_1 \text{ and } M E_2$

```
{ backpatch( $E_1$ .truelist, M.quad);
```

```
   $E$ .truelist:= $E_2$ .truelist;
```

```
   $E$ .falselist:=merge( $E_1$ .falselist, $E_2$ .falselist) }
```

# 布尔表达式的翻译模式

(3)  $E \rightarrow \text{not } E_1$

{  $E.\text{truelist} := E_1.\text{falselist};$   
 $E.\text{falselist} := E_1.\text{truelist}$ }

(4)  $E \rightarrow (E_1)$

{  $E.\text{truelist} := E_1.\text{truelist};$   
 $E.\text{falselist} := E_1.\text{falselist}$ }

# 布尔表达式的翻译模式

- (5)  $E \rightarrow id_1 \text{ relop } id_2$  {  $E.\text{truelist} := \text{makelist}(\text{nextquad})$ ;  
     $E.\text{falselist} := \text{makelist}(\text{nextquad}+1)$ ;  
    emit( 'j' relop.op ',' id<sub>1</sub>.place ',' id<sub>2</sub>.place ','  
        '0' );  
    emit( 'j, -, -, 0' ) }
- (6)  $E \rightarrow id$   
    {  $E.\text{truelist} := \text{makelist}(\text{nextquad})$ ;  
       $E.\text{falselist} := \text{makelist}(\text{nextquad}+1)$ ;  
      emit( 'jnz' ',' id.place ',' '-' ',' '0' );  
      emit( 'j, -, -, 0' ) }
- (7)  $M \rightarrow \varepsilon$  {  $M.\text{quad} := \text{nextquad}$  }

# $a < b$ or $c < d$ and $e < f$

100 (j<, a, b, 0)

101 (j, -, -, 102)

102 (j<, c, d, 104)

103 (j, -, -, 0)

104 (j<, e, f, 0) **true**list=100, 104

105 (j, -, -, 0) **false**list =103, 105

作为整个布尔表达式的"  
真""假"出口(转移目标)  
仍待回填

# 内容线索

- ✓ 中间语言
- ✓ 说明语句的翻译
- ✓ 赋值语句的翻译
- ✓ 布尔表达式的翻译
- 控制语句的翻译
- 过程调用的处理

# 控制语句的翻译

- 考虑下列产生式所定义的语句

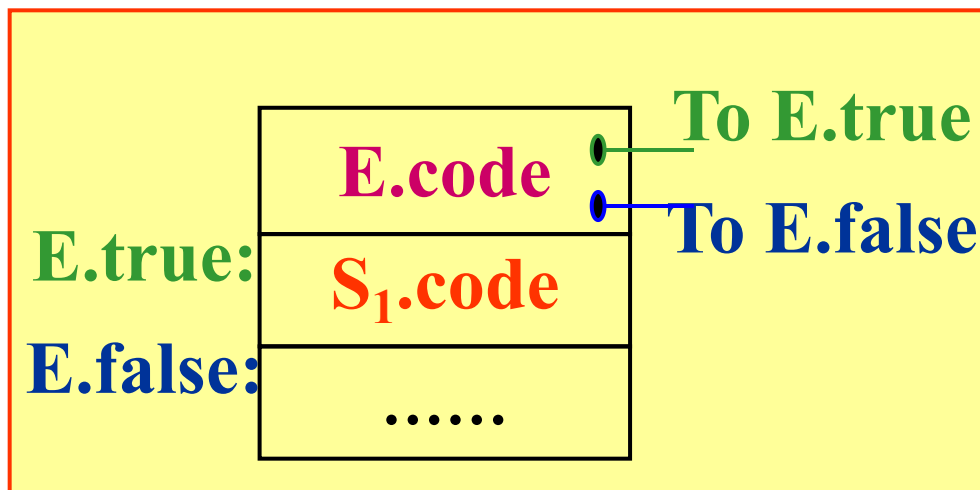
**$S \rightarrow \text{if } E \text{ then } S_1$**

**$\quad \quad \quad | \text{if } E \text{ then } S_1 \text{ else } S_2$**

**$\quad \quad \quad | \text{while } E \text{ do } S_1$**

其中**E**为布尔表达式。

# if-then语句的属性文法



产生式

$S \rightarrow \text{if } E \text{ then } S_1$

S.next之值是一个标号，  
它指出继S的代码之后  
将被执行的第一条三地址指令

语义规则

$E.\text{true} := \text{newlabel};$

$E.\text{false} := S.\text{next};$

$S_1.\text{next} := S.\text{next}$

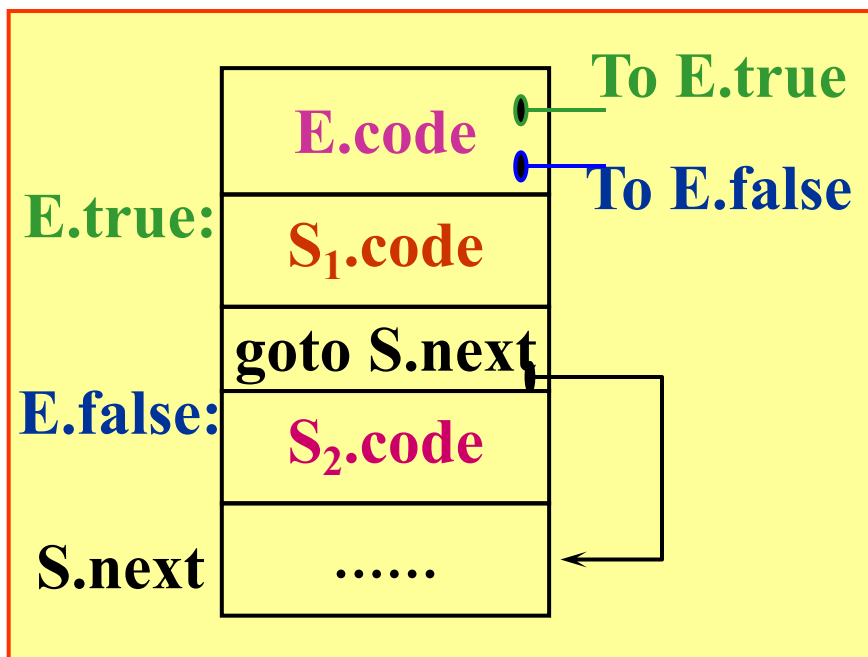
$S.\text{code} := E.\text{code} \parallel$

$\text{gen}(E.\text{true} \text{ ‘:’ }) \parallel S_1.\text{code}$

# if-then-else语句的属性文法

产生式

$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$



语义规则

```
E.true:=newlabel;  
E.false:=newlabel;  
S1.next:=S.next  
S2.next:=S.next;  
S.code:=E.code ||  
gen(E.true ':' ) || S1.code  
  
gen( 'goto' S.next) ||  
gen(E.false ':' ) || S2.code
```

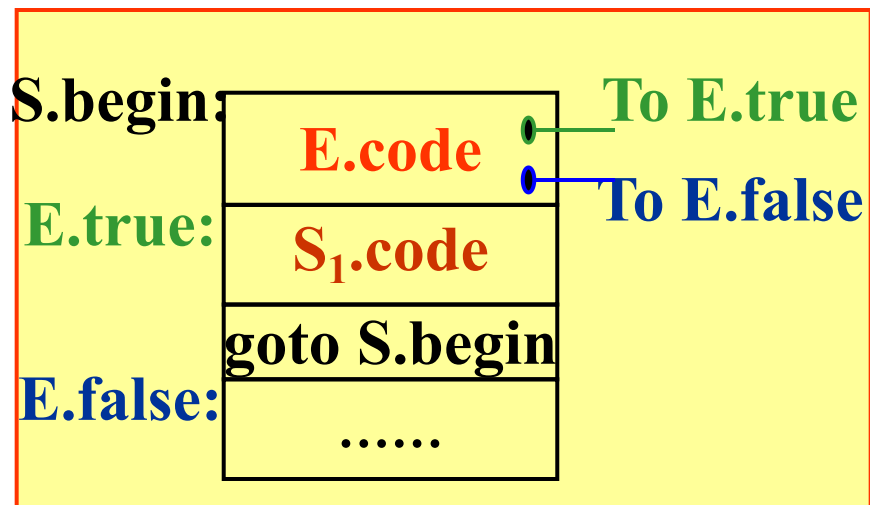
# while-do语句的属性文法

产生式

$S \rightarrow \text{while } E \text{ do } S_1$

语义规则

```
S.begin:=newlabel;  
E.true:=newlabel;  
E.false:=S.next;  
S1.next:=S.begin;  
S.code:=gen(S.begin ':' ) ||  
E.code ||  
gen(E.true ':' ) || S1.code ||  
gen( 'goto' S.begin)
```



考虑如下语句：

```
while a<b do
    if c<d then x:=y+z
    else x:=y-z
```

■ 生成下列代码：

L<sub>1</sub>:        if a<b goto L<sub>2</sub>

          goto Lnext

L<sub>2</sub>:        if c<d goto L<sub>3</sub>

          goto L<sub>4</sub>

L<sub>3</sub>:        T<sub>1</sub>:=y+z

          x:=T<sub>1</sub>

          goto L<sub>1</sub>

L<sub>4</sub>:        T<sub>2</sub>:=y-z

          x:=T<sub>2</sub>

          goto L<sub>1</sub>

Lnext:

E→id1 relop id2

E.code:=gen('if ' id1.place relop.op  
              id2.place 'goto' E.true)  
          ||gen('goto' E.false)

# 一遍扫描翻译控制流语句

- 考虑下列产生式所定义的语句:

(1)  $S \rightarrow \text{if } E \text{ then } S$

(2)  $\quad \quad | \text{if } E \text{ then } S \text{ else } S$

(3)  $\quad \quad | \text{while } E \text{ do } S$

(4)  $\quad \quad | \text{begin } L \text{ end}$

(5)  $\quad \quad | A$

(6)  $L \rightarrow L; S$

(7)  $\quad \quad | S$

- $S$ 表示语句,  $L$ 表示语句表,  
 $A$ 为赋值语句,  $E$ 为一个布尔表达式

# if 语句的翻译

相关产生式

$$\begin{aligned} S \rightarrow & \text{if } E \text{ then } S^{(1)} \\ & | \text{if } E \text{ then } S^{(1)} \text{ else } S^{(2)} \end{aligned}$$

改写后产生式

$$\begin{aligned} S \rightarrow & \text{if } E \text{ then } M S_1 \\ S \rightarrow & \text{if } E \text{ then } M_1 S_1 N \text{ else } M_2 S_2 \\ M \rightarrow & \epsilon \\ N \rightarrow & \epsilon \end{aligned}$$

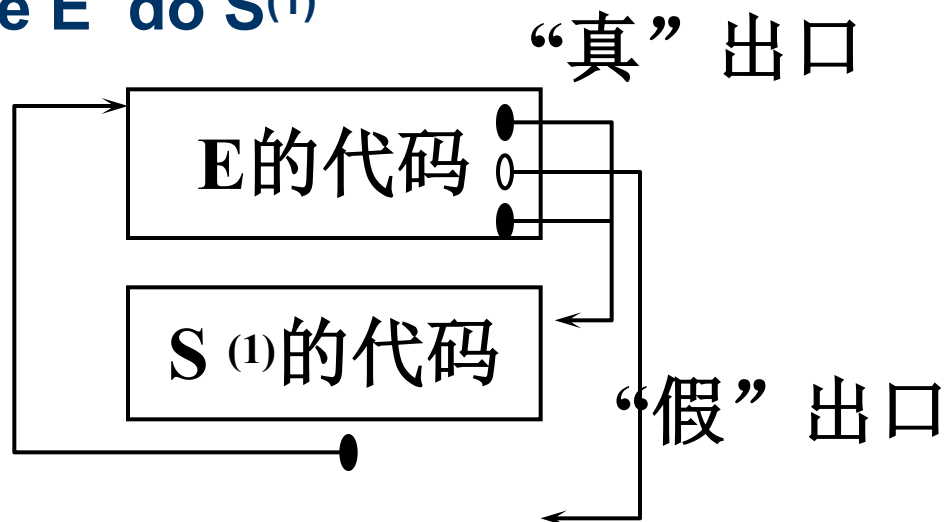
## 翻译模式:

1.  $S \rightarrow \text{if } E \text{ then } M \ S_1$   
{ backpatch(E.truelist, M.quad);  
**S.nextlist:=merge(E.falselist, S<sub>1</sub>.nextlist) }**
2.  $S \rightarrow \text{if } E \text{ then } M_1 \ S_1 \ N \ \text{else } M_2 \ S_2$   
{ backpatch(E.truelist, M<sub>1</sub>.quad);  
backpatch(E.falselist, M<sub>2</sub>.quad);  
S.nextlist:=merge(S<sub>1</sub>.nextlist, N.nextlist, S<sub>2</sub>.nextlist) }
3.  $M \rightarrow \varepsilon$                       { M.quad:=nextquad }
4.  $N \rightarrow \varepsilon$                       { N.nextlist:=makelist(nextquad);  
emit( 'j,—,—,—' ) }

# while 语句的翻译

相关产生式

$S \rightarrow \text{while } E \text{ do } S^{(1)}$



为了便于"回填", 改写产生式为:

$S \rightarrow \text{while } M1 \ E \text{ do } M2 \ S1$

$M \rightarrow \epsilon$

## 翻译模式:

1.  $S \rightarrow \text{while } M_1 \text{ E do } M_2 S_1$

    {backpatch( $S_1.\text{nextlist}$ ,  $M_1.\text{quad}$ );

      backpatch( $E.\text{truelist}$ ,  $M_2.\text{quad}$ );

$S.\text{nextlist} := E.\text{falselist}$

      emit( 'j, —, —, '  $M_1.\text{quad}$  ) }

2.  $M \rightarrow \varepsilon$       {  $M.\text{quad} := \text{nextquad}$  }

# 语句 $L \rightarrow L; S$ 的翻译

产生式

$$L \rightarrow L; S$$

改写为:

$$L \rightarrow L_1; M S$$

$$M \rightarrow \varepsilon$$

翻译模式:

- |                                |                                                                           |
|--------------------------------|---------------------------------------------------------------------------|
| 1. $L \rightarrow L_1; M S$    | { backpatch( $L_1.nextlist$ , $M.quad$ );<br>$L.nextlist := S.nextlist$ } |
| 2. $M \rightarrow \varepsilon$ | { $M.quad := nextquad$ }                                                  |

## 其它几个语句的翻译

$S \rightarrow \text{begin } L \text{ end}$   
 $\{ S.\text{nextlist} := L.\text{nextlist} \}$

$S \rightarrow A$   
 $\{ S.\text{nextlist} := \text{makelist}() \}$

$L \rightarrow S$   
 $\{ L.\text{nextlist} := S.\text{nextlist} \}$

# 翻译语句

**while (a<b) do  
if (c<d) then x:=y+z;**

**P195**

$S \rightarrow \text{if } E \text{ then } M \ S_1$   
 $\{ \text{backpatch}(E.\text{truelist}, M.\text{quad});$   
 $S.\text{nextlist} := \text{merge}(E.\text{falselist}, S_1.\text{nextlist}) \}$

$\text{nextquad});$

$\text{id}_2.\text{place} \text{ ' , ' '0' } );$

$M \rightarrow \varepsilon \ \{ M.\text{quad} := \text{nextquad} \}$

$S \rightarrow A \ \{ S.\text{nextlist} := \text{makelist}(\ ) \}$

$S \rightarrow \text{id}_1 = E \ \{ p := \text{lookup}(\text{id}_1.\text{name});$   
 $\text{if } p \neq \text{nil} \text{ then}$   
 $\text{emit}(p \text{ ' := ' } E.\text{place});$   
 $\text{else error} \}$

$E \rightarrow E_1 + E_2 \ \{ E.\text{place} := \text{newtemp};$   
 $\text{emit}(E.\text{place} \text{ ' + ' } E_1.\text{place} \text{ ' + ' } E_2.\text{place});$

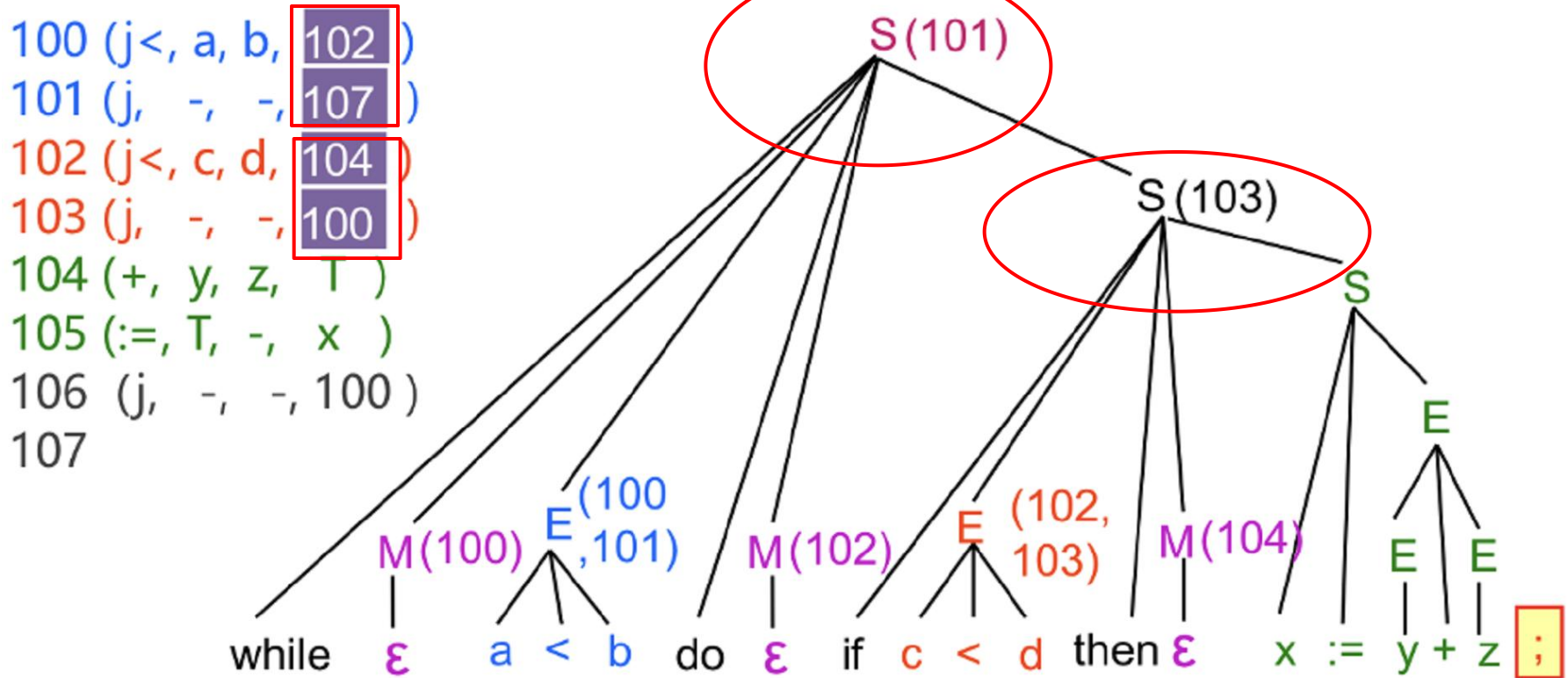
**P195**

$S \rightarrow \text{while } M_1 \ E \ \text{do } M_2 \ S_1$   
 $\{ \text{backpatch}(S_1.\text{nextlist}, M_1.\text{quad});$   
 $\text{backpatch}(E.\text{truelist}, M_2.\text{quad});$   
 $S.\text{nextlist} := E.\text{falselist}$   
 $\text{emit}(\text{ 'j, -, -, ' } M_1.\text{quad}) \}$

$M \rightarrow \varepsilon \ \{ M.\text{quad} := \text{nextquad} \}$

# 翻译语句

while (a<b) do  
if (c<d) then x:=y+z;



# 标号与goto语句

## ■ 标号定义形式

**L: S;**

当这种语句被处理之后，标号**L**称为“定义了”的。即在符号表中，标号**L**的“地址”栏将登记上语句**S**的第一个四元式的地址。

## ■ 标号引用

**goto L;**

向后转移:

```
L1: .....  
      .....  
      goto L1;
```

向前转移:

```
      goto L1;  
      .....  
L1:  .....
```

# 符号表信息

| 名字  | 类型  | ... | 定义否 | 地址  |
|-----|-----|-----|-----|-----|
| ... | ... | ... | ... | ... |
| L   | 标号  |     | 未   | r   |
|     |     |     |     |     |

(P) (j, -, -, 0)

...

(q) (j, -, -, p)

...

(r) (j, -, -, q)

产生式  $S' \rightarrow \text{goto } L$  的语义动作:

{ 查找符号表;

IF  $L$  在符号表中且"定义否"栏为"已"

THEN GEN( $J$ , -, -,  $P$ )

ELSE IF  $L$  不在符号表中

THEN BEGIN

把  $L$  填入表中;

置"定义否"为"未", "地址"栏为  $NXQ$ ;

GEN( $J$ , -, -, 0) END

ELSE BEGIN

$Q := L$  的地址栏中的编号;

置地址栏编号为  $NXQ$ ;

GEN( $J$ , -, -,  $Q$ )

END

}

## ■ 带标号语句的产生式:

**$S \rightarrow \text{label } S$**

**$\text{label} \rightarrow i:$**

## ■ **$\text{label} \rightarrow i:$** 对应的语义动作:

1. 若*i*所指的标识符(假定为*L*)不在符号表中, 则把它填入, 置"类型"为"标号", 定义否为"已", "地址"为**nextquad** ;
2. 若*L*已在符号表中但"类型"不为标号或"定义否"为"已", 则报告出错;
3. 若*L*已在符号表中, 则把标号"未"改为"已", 然后, 把地址栏中的链头(记为*q*)取出, 同时把**nextquad**填在其中, 最后, 执行 **BACKPATCH(*q*, nextquad)**。

# CASE语句的翻译

## ■ 语句结构

```
case E of
  C1: S1;
  C2: S2;
  ...
  Cn-1: Sn-1;
  otherwise: Sn
end
```

E是一个表达式，称为**选择子**。E通常是一个整型表达式或字符型变量。

## ■ 翻译法(一):

$T := E$

$L_1:$     **if**  $T \neq C_1$  **goto**  $L_2$   
           $S_1$ 的代码

**goto next**

$L_2:$     **if**  $T \neq C_2$  **goto**  $L_3$   
           $S_2$ 的代码

**goto next**

$L_3:$

...

$L_{n-1}:$     **if**  $T \neq C_{n-1}$  **goto**  $L_n$   
           $S_{n-1}$ 的代码

**goto next**

$L_n:$      $S_n$ 的代码

**next:**

## ■ 改进:

|          |           |
|----------|-----------|
| $C_1$    | $S_1$ 的地址 |
| $C_2$    | $S_2$ 的地址 |
| $\vdots$ | $\vdots$  |
| $E$      | $S_n$ 的地址 |

## ■ 翻译法(二):

计算E并放入T中

**goto test**

$L_1$ : 关于 $S_1$ 的中间码

**goto next**

...

$L_{n-1}$ : 关于 $S_{n-1}$ 的中间码

**goto next**

$L_n$ : 关于 $S_n$ 的中间码

**goto next**

**test: if  $T=C_1$  goto  $L_1$**

**if  $T=C_2$  goto  $L_2$**

...

**if  $T=C_{n-1}$  goto  $L_{n-1}$**

**goto  $L_n$**

**next:**

|           |                   |
|-----------|-------------------|
| $L_1$     | $L_1$ 的四元式首地址     |
| $L_2$     | $L_2$ 的四元式首地址     |
| $\vdots$  | $\vdots$          |
| $L_{n-1}$ | $L_{n-1}$ 的四元式首地址 |
| $L_n$     | $L_n$ 的四元式首地址     |

|           |           |
|-----------|-----------|
| $C_1$     | $P_1$     |
| $C_2$     | $P_2$     |
| $\vdots$  | $\vdots$  |
| $C_{n-1}$ | $P_{n-1}$ |

$P_i$ 是 $L_i$ 在  
符号表中  
的位置

(case,  $C_1$ ,  $P_1$ , -)

(case,  $C_2$ ,  $P_2$ , -)

...

(case,  $C_{n-1}$ ,  $P_{n-1}$ , -)

(case,  $T$ ,  $P_n$ , -)

(label, NEXT, -, -)

# 内容线索

- ✓ 中间语言
- ✓ 说明语句的翻译
- ✓ 赋值语句的翻译
- ✓ 布尔表达式的翻译
- ✓ 控制语句的翻译
- 过程调用的处理

# 过程调用的处理

- 过程调用主要对应两种事：
  - 传递参数
  - 转子（过程）
- 传地址：把实在参数的地址传递给相应的形式参数
  - 调用段预先把实在参数的地址传递到被调用段可以拿到的地方；
  - 程序控制转入被调用段之后，被调用段首先把实在参数的地址抄进自己相应的形式单元中；
  - 过程体对形式参数的引用与赋值被处理成对形式单元的间接访问。

# 过程调用的文法

- 过程调用文法:

- (1)  $S \rightarrow \text{call id (Elist)}$

- (2)  $\text{Elist} \rightarrow \text{Elist}, E$

- (3)  $\text{Elist} \rightarrow E$

- 参数的地址存放在一个队列中

- 最后对队列中的每一项生成一条par语句

# 过程调用的翻译

- 翻译方法：把实参的地址逐一放在转子指令的前面。

例如， **CALL S(A, X+Y)** 翻译为：

中间代码：计算**X+Y**，置于**T**中

**par A**     /\*第一个参数的地址\*/

**par T**     /\*第二个参数的地址\*/

**call S**     /\*转子\*/

## ■ 翻译模式

### 3. $Elist \rightarrow E$

{ 初始化 **queue** 仅包含 **E.place** }

### 2. $Elist \rightarrow Elist, E$

{ 将 **E.place** 加入到 **queue** 的队尾 }

### 1. $S \rightarrow \text{call id} (Elist)$

{ for 队列 **queue** 中的每一项 **p** do  
    **emit( 'param' p);**  
    **emit( 'call' id.place) }**

# 作业

■ P217

1,3,4,6,7